Code review workflows

Diffs, test results, and human judgment

Review is a human responsibility

Automation raises questions. A reviewer decides whether a change is correct, safe, maintainable, and in scope.

```
Issue and acceptance criteria
        ↓
Proposed change and checks
        ↓
Human review
        ↓
Accept, request revision, or implement manually
```

Review rubric

- Correctness against acceptance criteria
- Tests and check results
- Security, dependency, and permission implications
- Scope and maintainability
- Repository conventions and human approval

Conflict-resolution exercise

1. Read both branch intentions.
2. Write the merged behavior and tests before editing.
3. Resolve the conflict manually in the sandbox.
4. Run checks and request review.

Review the same diff through separate passes

1. Record the human verdict before assisted review.
2. Leave the patch uncommitted and run `/review`.
3. Run `/security-review` against that same local patch.
4. Triage both outputs, then commit and request Copilot PR review.
5. Repeat the local commands on the correction diff before committing fixes.

Keep the original verdict. Record no-findings results.

A finding needs support

Claim	Check
Exposed password field	Inspect the actual response and add a regression test
Database compromise	The mock route never executes SQL; reject that claim
Leaked credential	The strings are fake and unused; reject a live-leak claim
Pagination defect	Check page-one behavior separately from security review

Record: review source, file/line, impact, check, and triage decision.

Security review has a narrow scope

`/security-review` examines current local changes. It's read-only and doesn't replace repository security scanning.

A clean result doesn't make a change secure. The human reviewer decides.

Agenda and time plan

Time	Topic
0:00	Review workflow and roles
0:08	Summaries and descriptions
0:18	Local commands and PR review
0:28	Defect and triage demo
0:43	Suggestions and conflicts
0:51	Limits and adoption
0:58	Lab handoff and Q&A

Copilot across a PR lifecycle

```
Issue and acceptance criteria
          ↓
Branch and bounded commits
          ↓
Draft summary and PR description
          ↓
Automated checks + Copilot first pass
          ↓
Human review and author response
          ↓
Approval, revision, or close
```

Commit messages and PR descriptions

Copilot drafts text from the visible change. Authors check that:

- the user-visible behavior is accurate;
- tests and results aren't invented;
- breaking changes and migrations are named;
- security and dependency implications are named;
- issue links and acceptance criteria match the work.

A polished summary can still be wrong.

A reviewable PR template

```
## Problem
What user-visible problem does this solve?

## Change
List the bounded implementation choices.

## Validation
- [ ] Focused tests
- [ ] Existing checks
- [ ] Manual edge case

## Risk and rollback
Name data, dependency, security, and recovery considerations.
```

Requesting Copilot review

1. Check that the PR is ready for a first pass.
2. Request the Copilot review from the PR.
3. Treat the comments as hypotheses.
4. Resolve them, reject them with a reason, or implement a checked fix.
5. Request human review with the diff, findings, and check results.

Copilot review doesn't replace required reviewers.

Automatic review configuration

You can configure automatic review at repository or organization scope.

Before enabling:

- choose eligible repositories;
- decide who triages comments;
- keep branch protection and required human approvals;
- watch noise, missed findings, and review delay;
- know how to turn it off.

Quality gates

Gate	Checks
Ready for first pass	Clear description, focused diff, tests available
Ready for human review	Checks pass; AI comments triaged
Ready for approval	Acceptance criteria met; risks reviewed
Ready for merge	Required approvals and branch protections satisfied

Copilot review comes before human approval. It never replaces it.

Demo PR: intentional defect

```
export function canViewInvoice(user, invoice) {
  return user.role === "admin" || user.id === invoice.ownerId;
}
```

The requirement says suspended users must never view invoices.

```
if (user.status === "suspended") return false;
return user.role === "admin" || user.id === invoice.ownerId;
```

Ask which tests should accompany this suggestion.

Demo narrative: complete lifecycle

1. Open the issue and read acceptance criteria aloud.
2. Inspect the focused branch diff before generated prose.
3. Generate or improve the PR description; correct unsupported claims.
4. Run required checks and request Copilot review.
5. Classify each comment: valid, uncertain, noise, or out of scope.
6. Apply one checked fix and add a regression test.
7. Request human review and record the final decision.

Reviewing suggested fixes

Before applying:

- restate the finding in your own words;
- reproduce the defect or find the violated requirement;
- inspect the complete patch, not only highlighted lines;
- add a test that fails before the fix;
- run broader checks in proportion to the risk.

One-click application saves typing. The reviewer still decides.

What review may catch

- localized correctness issues;
- missing error handling or tests;
- suspicious patterns visible in the diff;
- inconsistent naming or repository conventions.

What it may miss

- unstated product intent;
- cross-system behavior absent from context;
- runtime, operational, or legal risk;
- subtle authorization and data-boundary flaws;
- plausible but misleading changes.

Merge conflict resolution

```
Base behavior + branch A intent + branch B intent
                          ↓
              write merged acceptance criteria
                          ↓
                resolve conflict in sandbox
                          ↓
                 run tests and human review
```

Don't blindly choose "ours" or "theirs."

Adoption patterns

Start: Optional first-pass review on a few repositories.

Measure: Actionable findings, false positives, review time, rework, and defects that reach users.

Adapt: Tune scope, guidance, and escalation.

Expand: Only when quality and developer experience hold up.

Know how to pause reviews for a repository that needs an exception.

Facilitation prompts

- Which comment needs domain expertise to evaluate?
- Which code reference or test would resolve an uncertain comment?
- Which repositories should not use automatic review?
- How will the team detect over-reliance?

Lab handoff

You will:

1. follow one pull request from a flawed change brief;
2. classify human and Copilot findings;
3. apply only fixes supported by code references or test results;
4. resolve the prepared conflict without losing either branch goal;
5. record the final human decision.

Deliverable: Review notes, the corrected project, conflict results, the initial and correction outputs from `/review` and `/security-review`, and the private training PR reference.

Remember

1. Check generated summaries against the diff.
2. AI review is a first pass. A human decides.
3. Suggested fixes need reproduction, inspection, and tests.
4. Conflict resolution starts from the intended merged behavior.
5. Measure adoption and keep it reversible.

Questions

Which review decision needs a human, and what should they check?